

UNIVERSITÀ DEGLI STUDI DI TORINO
Dipartimento di Fisica

**REPORT
RETI NEURALI**

RICONOSCIMENTO DI SEGNALI STRADALI

A CURA DI
MARCO GORGONE

ANNO 2025/2026

Indice

1	Problema: Riconoscimento dei Segnali Stradali	3
1.1	Dataset	4
2	Metodo	6
2.1	Rete Utilizzata	6
2.2	Funzione di Loss	6
2.3	Valutazione	9
3	Analisi	10
3.1	Scelta del Modello	10
3.2	ROC Curve	11
3.3	Calibrazione del Momento	11
4	Codice	14
4.1	Caso Reale	15
4.1.1	Divisione	15
4.1.2	YOLO	15
5	Conclusione	17

1 Problema: Riconoscimento dei Segnali Stradali

In questa era dell'Intelligenza Artificiale, gli esseri umani stanno diventando sempre più dipendenti dalla tecnologia. Con l'evoluzione tecnologica, aziende multinazionali come Google, Tesla e molte altre stanno lavorando per automatizzare i veicoli, cercando di creare mezzi autonomi il più possibile precisi. Infatti, le auto a guida autonoma, in cui è il veicolo stesso a simulare il conducente, non necessitano di una guida umana. È lecito preoccuparsi degli aspetti legati alla sicurezza, in quanto è sempre presente la possibilità di incidenti; sappiamo infatti che nessuna macchina è ancora in grado di eguagliare la precisione degli esseri umani alla guida. I ricercatori stanno testando numerosi algoritmi per garantire una sicurezza stradale ottimale. Fra questi, un compito fondamentale è il **Riconoscimento dei Segnali Stradali**.

Quando si è in strada, si vedono vari segnali stradali come curve a sinistra o a destra, limiti di velocità, divieto di sorpasso ai veicoli pesanti, divieto di accesso, attraversamento pedonale per bambini, ecc., che è necessario seguire per una guida sicura. Allo stesso modo, i veicoli autonomi devono interpretare questi segnali e prendere decisioni per garantire la precisione. Per riconoscere a quale categoria appartiene un segnale stradale è quindi necessario classificarlo correttamente.

In questo lavoro viene proposto un modello di classificazione dei segnali stradali basato su una rete neurale convoluzionale (CNN).

In figura 1 è presente un esempio visivo che descrive il problema.

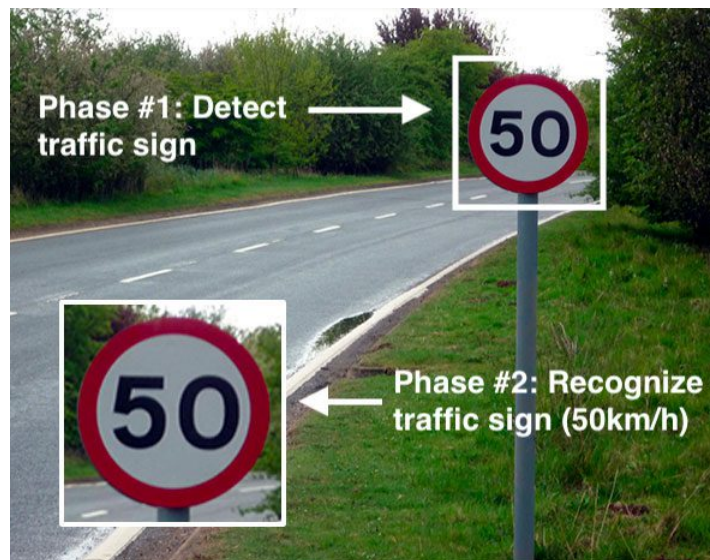


Figura 1: Esempio

1.1 Dataset

Il dataset completo delle immagini, scaricabile al link <https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>, è composto da più di 50.000 foto di vari segnali stradali (limiti di velocità, attraversamenti, segnali stradali, ecc.), con una dimensione totale di circa 560 MB.

Per questo lavoro nello specifico sono state utilizzate le immagini contenute nel percorso DITS, con una dimensione di 450 MB.



Figura 2: Esempi visuali dei dati

Del dataset sono presenti due versioni una in cui i segnali sono divisi per segnale specifico e un'ultima da me scelta la divisione per macro categoria di segnale, in questo modo i segnali del dataset si dividono nelle loro 3 macrocategorie, a cui è poi stata aggiunta attraverso uno script una quarta categoria, arrivando ad avere le seguenti categorie:

- Divieto
- Pericolo
- Obbligo
- Background (utile per un secondo scopo)

2 Metodo

Verrà esaminato ora l'approccio eseguito per risolvere la problematica: il compito è configurato come un problema di classificazione con quattro categorie (indicazione, avvertimento, divieto, background).

2.1 Rete Utilizzata

In merito a ciò, dopo aver condotto alcuni esperimenti, i quali verranno descritti più dettagliatamente nel capitolo successivo, si è optato per l'implementazione di una Convolutional Neural Network (CNN) ispirata ad AlexNet (*LeNetColor*). A questa architettura, è stata applicata sia la tecnica della batch normalization che il dropout al fine di prevenire problemi di overflow durante il calcolo delle funzioni di attivazione e problemi di overfitting. Da adesso in poi il modello verrà nominato come **MiniAlexNetV2**. Nella figura 3 è possibile osservare una rappresentazione schematica della rete LenetColor che è il punto di partenza e la prima modifica eseguita per ottenere la rete MiniAlexNet, che sono state entrambe il punto di partenza per poi arrivare alla rete MiniAlexNetV2 in figura 4, la quale è stata poi scelta come rete finale.

Negli input della rete, sono state adottate le cosiddette "regole generali approssimative": i batch di dati sono stati configurati con dimensioni di 1024 per la fase di addestramento e di 2048 per le fasi di convalida e test, implementando MaxPooling con dimensione 2 e uno stride di dimensione 1. Come funzione di attivazione, è stata impiegata l'Unità Lineare Rettificata, nota come ReLU, la quale restituisce il valore stesso della combinazione se i dati sono positivi, mentre annulla quelli negativi. Tutti questi parametri sono formalmente espressi mediante un'apposita equazione matematica (1):

$$f(x) = \begin{cases} 0 & \text{se } x < 0 \\ x & \text{se } x > 0 \end{cases} \quad (1)$$

2.2 Funzione di Loss

La funzione di Loss utilizzata è la *Cross Entropy Loss* descritta in (2):

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \cdot \log(\hat{y}_{ij}) \quad (2)$$

dove:

N è il numero di campioni,

C è il numero di classi,

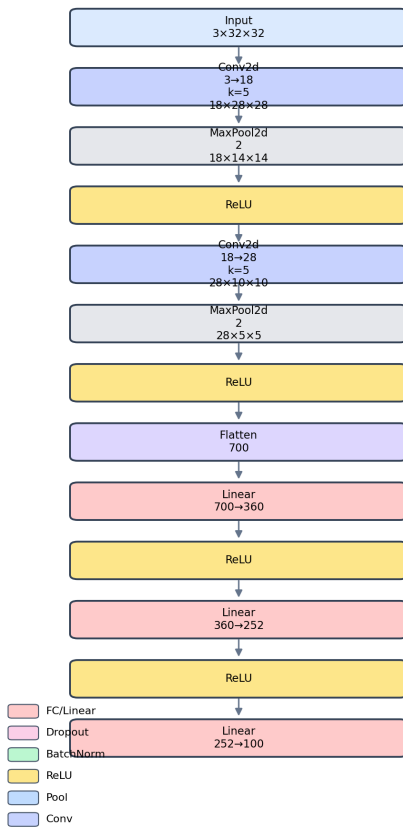
y_{ij} è la vera probabilità della classe j per il campione i ,

$\hat{y}_{ij}$ è la probabilità predetta della classe j per il campione i .

La formula della cross entropy loss, la quale consente il confronto tra due distribuzioni di probabilità, risulta fondamentale in quanto la rete **MiniAlexNetV2** fornisce per ciascuna immagine una probabilità associata all'appartenenza a una specifica classe. È pertanto necessario effettuare un confronto tra questa distribuzione di probabilità

LeNetColor

Schema architetturale verticale



MiniAlexNet

Schema architetturale verticale

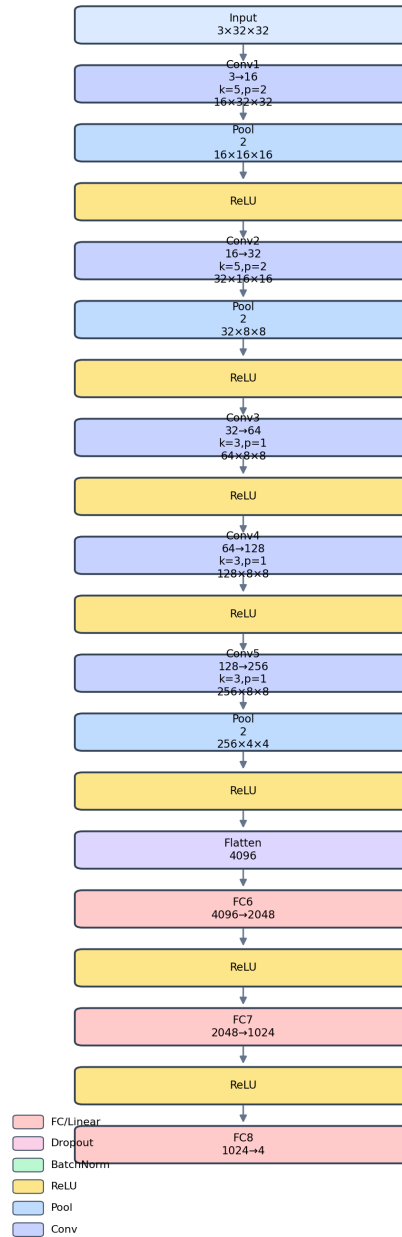


Figura 3: Confronto tra le tre reti neurali

MiniAlexNetV2

Schema architetturale verticale

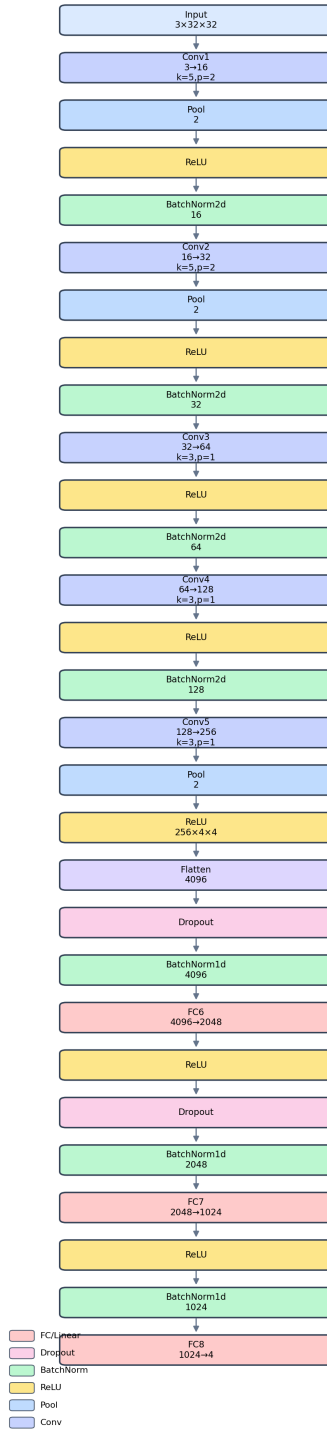


Figura 4: MiniAlexNetV2, rete utilizzata per il training

predetta e la distribuzione "reale", la quale assume un valore diverso da zero solamente per la classe alla quale l'immagine in input appartiene.

In termini formali, la funzione di Loss che è stata utilizzata presenta una leggera variazione rispetto a quella precedentemente menzionata. L'equazione adottata è la (3):

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_i^N (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)) \quad (3)$$

dove:

N è il numero di classi,

y_i è la vera probabilità della classe i ,

$\hat{y}_i$ è la probabilità predetta della classe i .

2.3 Valutazione

La validità delle previsioni è stata valutata attraverso il parametro di accuratezza. Tale parametro è stato derivato definendo una matrice di confusione elementare, come indicato nella tabella 1:

	Realtà	
	Positivo	Negativo
Previsioni Positivo	Vero Positivo	Falso Positivo
Previsioni Negativo	Falso Negativo	Vero Negativo

Tabella 1: Matrice di Confusione

Da questa analisi, è possibile definire l'accuratezza del modello come il rapporto tra il numero di previsioni corrette e il numero totale di previsioni, come indicato nell'equazione (4):

$$\text{Accuracy} = \frac{\text{Numero di Previsioni Corrette}}{\text{Numero Totale di Previsioni}} \quad (4)$$

Questo parametro fornisce una misura della corretta classificazione effettuata dal modello rispetto al numero complessivo di previsioni eseguite.

Oltre all'accuratezza, è stata elaborata anche la curva ROC, una curva grafica che illustra le prestazioni di un modello di classificazione al variare della soglia di decisione.

3 Analisi

Inizialmente sono state condotte una serie di analisi allo scopo principale di identificare il modello che ha restituito i risultati più promettenti. Successivamente, ci si è dedicati all'indagine delle configurazioni ottimali del momento, al fine di massimizzare l'accuratezza.

3.1 Scelta del Modello

Per identificare il modello ottimale, sono stati condotti diversi approfondimenti. Per il primo approfondimento è stata impiegata una rete nota come **LeNetColor**, successivamente una rete basata su AlexNet, denominata **MiniAlexNet**, ed infine, dopo aver introdotto sia layer di Dropout che layer di Batch Normalization, abbiamo ottenuto la versione **MiniAlexNetV2**.

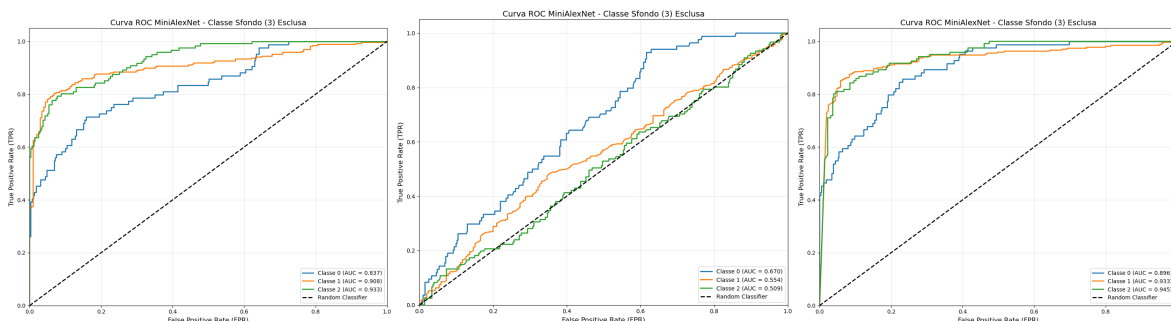


Figura 5: Confronto tra le ROC curve delle tre reti

L'esame della Figura 7 evidenzia in modo netto come la rete MiniAlexNetV2 raggiunga un'accuratezza notevolmente superiore rispetto alla MiniAlexNet, che mostra, al contrario, un comportamento oscillante. Sebbene la AlexNet presenti un andamento migliore rispetto alla MiniAlexNet, essa rimane comunque inferiore alla MiniAlexNetV2. Pertanto, dal punto di vista dell'accuratezza, la rete MiniAlexNetV2 emerge come la più efficace.



Figura 6: Comparazione tra curve loss

Dall'analisi della Figura 6, emergono notevoli differenze tra le reti MiniAlexNetV2, MiniAlexNet, e AlexNet, le quali sono state addestrate rispettivamente per 200 epoche durante la fase di training. La rete MiniAlexNetV2 mostra valori di loss inferiori rispetto alle altre due reti.

Tuttavia, durante la fase di test, la curva della MiniAlexNetV2 mostra inizialmente un picco, indicativo di una probabile fase di overfitting nelle prime epoche. Successivamente, la curva si stabilizza su valori di loss più contenuti. Questo comportamento sottolinea la complessità dell'addestramento e la necessità di valutare attentamente le dinamiche della loss durante l'intero processo di addestramento e test.

Questi risultati hanno indotto la concentrazione degli sforzi sull'ulteriore sviluppo della rete MiniAlexNetV2.

I parametri utilizzati per queste prove sono stati:

- Tasso di apprendimento (*Learning Rate*): 0.0001
- Momento (*Momentum*): 0.6

3.2 ROC Curve

Trovati quindi modello e momento, come mostrato prima, è stato reiterato il processo di training, allenando le reti per altre 200 epoche e calcolando la ROC Curve, come mostrato in Figura 7. La curva ROC, che rappresenta la relazione tra il tasso di veri positivi (TPR) e il tasso di falsi positivi (FPR) al variare della soglia di classificazione, è stata utilizzata per valutare le prestazioni del modello in modo più dettagliato.

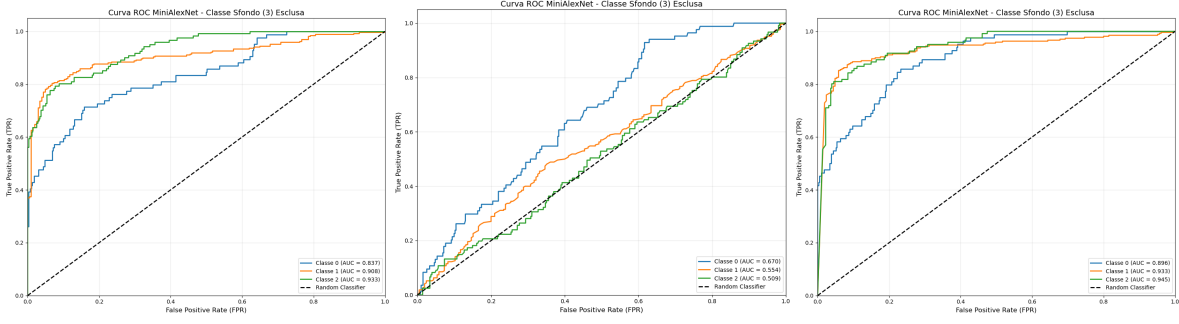


Figura 7: Confronto tra le ROC curve delle tre reti

3.3 Calibrazione del Momento

Una volta definito il modello, sono stati dedicati sforzi all'ottimizzazione del momento al fine di calibrarlo al meglio. Per valutare l'efficacia di diverse opzioni di momento, ho considerato solo quale tra i valori proposti conducesse a un aumento dell'accuratezza congiuntamente a una convergenza della loss verso valori inferiori.

I dati sull'accuratezza e sulla loss durante l'addestramento del precedente esperimento con MiniAlexNetV2 sono stati conservati. Successivamente, la rete è stata nuovamente addestrata utilizzando un valore di momento pari a 0.7. I risultati di questo test sono illustrati nella Figura 8.

Come evidenziato, il modello presenta un'accuratezza leggermente superiore durante la fase di addestramento e test quando il momento è impostato a 0.6. Le curve di loss convergono a valori leggermente inferiori e seguono un andamento poco più regolare. Sulla base di tali risultati, si è proceduto mantenendo un valore di momento pari a 0.6.

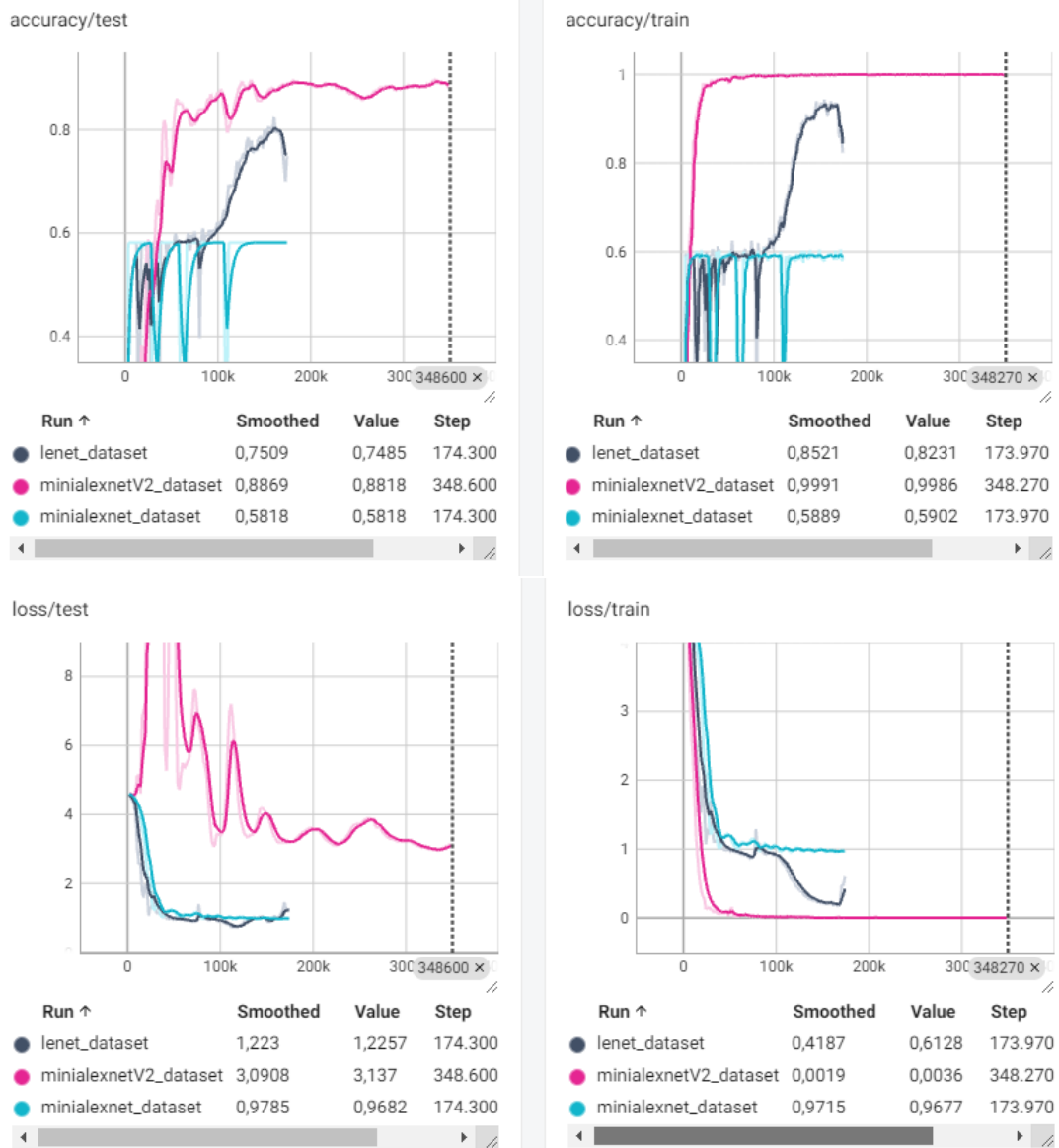


Figura 8: Comparazione tra curve accuracy sopra, tra curve loss sotto.

4 Codice

Il codice da me utilizzato è stato suddiviso e strutturato in modo tale da avere all'interno della sotto cartella "python_file" i seguenti file:

- Il file denominato *dataclass* che definisce le classi e le trasformazioni per la manipolazione di dataset di segnali stradali. Include classi per il training (*StreetSignDataset*) e il testing (*StreetSignTest*), insieme a trasformazioni come il ridimensionamento (*Rescale*), il ritaglio casuale (*RandomCrop*), il ritaglio specifico (*Crop*), e la conversione in tensori (*ToTensor*). La funzione *show_landmarks_batch* è un'utilità per visualizzare batch di immagini con landmarks. In sintesi, il codice fornisce un'infrastruttura per preparare dati e modelli per immagini di segnali stradali.
- Il file denominato *function* che fornisce funzioni compatte per l'addestramento e il test di un classificatore. La prima parte del codice addestra un modello utilizzando SGD e cross-entropy, con TensorBoard e salvataggio dei pesi. La seconda parte valuta le prestazioni del modello su un set di dati di test, restituendo predizioni e etichette reali. In sintesi il flusso principale configura parametri, carica il modello e itera attraverso le epoche e le fasi di training/test, registrando i risultati su TensorBoard e salvando i pesi del modello.
- Il file denominato *network* implementa l'architettura delle reti neurali testate, con variazioni nelle architetture e nell'utilizzo di tecniche come Batch Normalization e Dropout per migliorare le prestazioni e la generalizzazione del modello.
- Il file denominato *detect_and_classify* utilizzato per implementare la funzione che permette di usare YOLO allo scopo di riconoscere i segnali stradali all'interno di un'immagine presa da un contesto reale. Sarà quindi questa parte che si occuperà di prendere le immagini passarle alla rete YOLO preallenata con immagini reali e infine la porzione di immagine tagliata col segnale stradale viene passata alla rete MiniAlexNetV2 per identificare che tipo di segnale sia, una volta rintracciato esso

Infine sono stati creati dei notebook con Jupyter in cui è stata usata la nostra struttura dati, insieme alle funzioni di test e training per poter allenare il nostro modello, convertendo prima le immagini da inserire nella struttura dati e modificandole con le opportune trasformazioni. I notebook in questione sono:

- *train* si occupa dell'allenamento delle reti *AlexNet*, *MiniAlexNet* e *MiniAlexNetV2* che si occupano di convertire le immagini in tensori che possano allenare la rete, e che infine ritornano il valore di accuracy per ogni tipo di rete testata.
- *result* che opera caricando i tre modelli pre-allenati. Successivamente, utilizza l'URL di un'immagine o un'immagine locale per eseguire una previsione sulla categoria del segnale stradale presente nell'immagine. La categoria predetta viene successivamente stampata, comprendendo classi come *indicatory* (indicazione), *prohibitory* (divieto), *warning* (pericolo), oppure segnalando ERROR 404 NOT FOUND in caso di errore nella previsione il caso *Background* con cui abbiamo allenato la rete qui non lo consideriamo, anche perchè non è una classe di interesse.

- *test_reale* che opera caricando solo il modello *MiniAlexNetV2*, usandolo per eseguire una previsione sulla categoria del segnale stradale presente in un'immagine reale, e infine stampando la categoria predetta. In questo caso, l'immagine reale viene processata per identificare e ritagliare il segnale stradale, che viene poi passato alla rete *MiniAlexNetV2* per la classificazione. La categoria predetta viene poi inserita sull'immagine e infine la nuova immagine stampata.

4.1 Caso Reale

Analizziamo più in dettaglio il notebook *test_reale* in cui è presente la parte di codice relativa al test della rete neurale in un contesto urbano. Se, per esempio, dotassimo un veicolo di una telecamera per riprendere la strada in tempo reale, potremmo acquisire delle immagini durante il tragitto, e verificare la presenza di segnali stradali inviando infine l'output al sistema di bordo o al guidatore. Per fare ciò, ci si è posti il problema di come isolare la singola immagine del segnale rispetto al contesto; per eseguire questa ricerca si sono usati due processi:

- **Divisione in sottoimmagini:** consiste nel suddividere l'immagine in numerose porzioni e verificare per ognuna di esse il contenuto;
- **Uso di YOLO:** utilizzo di uno strumento di object detection che permette di individuare le regioni d'interesse (ROI) all'interno delle immagini, identificando le porzioni candidate alla classificazione;

4.1.1 Divisione

L'uso della divisione in sottoimmagini è sembrata inizialmente la soluzione più semplice, ma ha presentato scarsi risultati sia in termini di prestazioni che di affidabilità. Il motivo risiede nella necessità di processare sequenzialmente ogni ritaglio, causando lentezza di calcolo. Inoltre, l'accuratezza è limitata dal fatto che il dataset di addestramento non copre tutte le possibili variazioni del contesto circostante.

4.1.2 YOLO

Per ovviare ai limiti della suddivisione manuale, nel notebook *test_reale* è stato implementato l'uso dell'algoritmo **YOLO** (You Only Look Once). A differenza dell'approccio precedente, YOLO esegue l'object detection in un unico passaggio, identificando istantaneamente le coordinate dei segnali stradali (*Bounding Boxes*) all'interno di scene complesse.

Il flusso di lavoro implementato prevede i seguenti passaggi:

1. Caricamento dell'immagine ad alta risoluzione catturata nel contesto urbano.
2. Applicazione del modello YOLO per individuare le regioni d'interesse (ROI) corrispondenti ai segnali.
3. Ritaglio (*cropping*) automatico delle aree individuate.
4. Passaggio dei ritagli alla rete **MiniAlexNetV2** per la classificazione finale della macro-categoria.

Questo approccio ibrido ha permesso di ottenere una precisione estremamente elevata, poiché la rete di classificazione lavora solo su porzioni di immagine già identificate come "segnali", eliminando il rumore di fondo del contesto cittadino e riducendo drasticamente i tempi di calcolo rispetto alla scansione sequenziale.

5 Conclusione

Le conclusioni raggiunte suggeriscono che il modello proposto è in grado di affrontare con successo il compito assegnato, con possibilità di ulteriori miglioramenti. A questo proposito, si suggerisce di:

- Aumentare la quantità e la diversità dei dati di addestramento: utilizzare un set di dati più ampio e vario per migliorare la capacità di generalizzazione della rete.
- Sperimentare con diverse architetture di reti neurali: testare modelli con architetture diverse per identificare la più adatta al compito di riconoscimento dei segnali stradali.
- Utilizzare tecniche di regolarizzazione per ridurre l'*overfitting*: implementare tecniche di regolarizzazione per evitare che la rete si adatti eccessivamente ai dati di addestramento.
- Ottimizzare gli *hyperparametri*: regolare parametri come il tasso di apprendimento e la dimensione del batch per massimizzare le prestazioni del modello.
- Esplorare la quantizzazione dei pesi: ridurre la complessità della rete attraverso la quantizzazione dei pesi, mantenendo comunque un livello accettabile di accuratezza.
- Analizzare le attivazioni nei vari strati della rete: approfondire la comprensione del processo di apprendimento esaminando le attivazioni nei vari strati della rete neurale.

Per riassumere, questa esperienza non rappresenta solo un punto di arrivo, piuttosto un punto di partenza per ulteriori esplorazioni nel campo del riconoscimento dei segnali stradali attraverso l'impiego di modelli di machine learning. L'apertura a nuove idee, l'iterazione continua e la volontà di esplorare nuovi approcci saranno fondamentali per affrontare le sfide emergenti e migliorare costantemente le prestazioni dei modelli proposti.

Riferimenti bibliografici

- [1] *S. Battiato, G. Signorello, G.M. Farinella, F. Ragusa, A. Furnari. Ego-ch: Dataset and fundamental tasks for visitors behavioral understanding using egocentric vision. 2019.*
- [2] *G.M Farinella, A. Furnari. Diapositive del corso di Machine Learning anno 2022/2023.*